

Цель работы: получить опыт выпуска и подписи самоподписанных сертификатов и использование доменного имени.

Теоретическая справка

DNS — это система, которая сопоставляет доменное имя с IP-адресом.

Цифровой сертификат — это файл, который связывает публичный ключ с доменным именем и подтверждает, кому этот ключ принадлежит. Он подписывается центром сертификации и позволяет клиенту установить защищённое TLS-соединение.

Цепь доверия — это последовательность сертификатов от серверного сертификата до корневого центра сертификации, которому доверяет система. Если каждый сертификат в цепочке корректно подписан и корневой находится в хранилище доверенных, соединение считается безопасным.

Порядок выполнения работы



Копайтесь сами

Для максимально эффективного усвоения учебного материала рекомендуется использовать подсказки только после тщательного обдумывания. При возникновении вопросов не стесняйтесь обращаться в чат.

Подготовка



Экономьте грант

Выключайте виртуальную машину в случае перерыва в выполнении работы или после завершения.

1. Создайте ssh-ключ по гайду [S Как сгенерировать SSH-ключ... @Академия Selectel](#). Если у вас уже есть, пропускайте пункт.
2. Создайте виртуальную машину Selectel (подробнее смотрите практику 1).



Тюнинг безопасности

В энтерпрайзе после поднятия виртуальной машины нужно подкорректировать настройки ядра и sshd для повышения безопасности. Для данной лабораторной работы мы пропустим эти шаги, к тому же некоторые бестпрактик настроил облачный провайдер.

3. Подключитесь к машине `ssh root@(публичный IP машины)`.
4. Получите бесплатный домен на сервисе <https://www.duckdns.org/> и привяжите его к публичному IP виртуальной машины.
5. Придумайте, как предусмотреть вариант, если у виртуальной машины сменится белый IP-адрес?
 - ✓ Подсказка
Напишите cron, который будет интервально выполнять скрипт, посылающий запрос на API Duck DNS.
 - ✓ Вторая подсказка
В скрипте выполняется curl -s на endpoint Duck DNS, который принимает GET-параметр — ваш токен Duck DNS. Вывод перенаправьте в /dev/null.

Создание Root CA и CSR



Не для продакшена

Использование самоподписанных сертификатов практически не встречается в энтерпрайзе. Примитивный метод в лабораторной работе показан для осознания концепций. В ЛР №2 будет показан более предпочтительный метод выпуска сертификатов с Let's Encrypt.

1. Сгенерируйте корневой сертификат и секретный ключ (это одна команда).
 - В CN укажите свою фамилию и первые буквы имени и отчества на латинице.
 - Используйте алгоритм хэширования SHA 256.
 - Ключ создавайте RSA длиной 2048 бит.
 - Срок — 10 лет

✓ Подсказка

```
openssl req -x509 -sha256 -days 3650 -newkey rsa:2048 -keyout root_sahno_ca.key -out root_sahno_ca.crt -subj "/CN=CA_SAHNOYA"
```

2. Импортируйте корневой сертификат.

✓ Подсказка

```
cp root_sahno_ca.crt /usr/local/share/ca-certificates/root_sahno_ca.crt
```

```
update-ca-certificates
```

3. Сгенерируйте ключ RSA длиной 2048 бит для своего будущего примитивного сайта.

✓ Подсказка

```
openssl genrsa -out site.key 2048
```

4. Создайте Certificate Signing Request (CSR) с использованием ключа из прошлого пункта. В CN укажите Ваш домен.

✓ Подсказка

```
openssl req -new -key site.key -out site.csr -subj "/CN=sakhnoya.duckdns.org"
```



CN устарел, SAN важнее

Это допущение для лабораторной работы.

Подпись CSR

- Передайте CSR на подпись любому вашему однокурснику, который выполнил прошлые разделы. Не используйте мессенджер для передачи файла. Подумайте, как передать CSR через <https://ru.wikipedia.org/wiki/Netcat> на виртуальную машину однокурсника. Вам должны отдать подписанный и корневой сертификаты.

- Импортируйте корневой сертификат одногруппника в систему.
- Проверьте сертификат

✓ Подсказка

```
openssl verify incoming_signed_request.crt
```

- Если вам передали CSR, подпишите его на один год с использованием SHA 256. Отдайте однокурснику подписанный сертификат и корневой сертификаты через netcat.

✓ Подсказка

```
openssl x509 -req -in incoming_request.csr -CA root_ca.crt -CAkey root_ca.key -CAcreateserial -out sakhnoya_signed_by_ivanova.crt -days 365 -sha256
```

Запуск приложения

1. Создайте файл [app.py](#) со следующим содержимым:

```
import http.server
import socketserver
import ssl
import os
```

```
# Заполнить значения
STUDENT_SURNAME = ""
STUDENT_NAME = ""
CERT_FILE = ""
KEY_FILE = ""

PORT = 443

class MyHandler(http.server.SimpleHTTPRequestHandler):
    def do_GET(self):
        self.send_response(200)
        self.send_header("Content-type", "text/plain; charset=utf-8")
        self.end_headers()

        response_text = (
            f"Это виртуальная машина студента: {STUDENT_SURNAME} {STUDENT_NAME}\n"
            f"IP-адрес клиента: {self.client_address[0]}\n"
        )

        self.wfile.write(response_text.encode("utf-8"))

httpd = socketserver.TCPServer(("", PORT), MyHandler)

context = ssl.SSLContext(ssl.PROTOCOL_TLS_SERVER)
context.load_cert_chain(certfile=CERT_FILE, keyfile=KEY_FILE)

httpd.socket = context.wrap_socket(httpd.socket, server_side=True)

print(f"HTTPS сервер запущен на порту {PORT}")

httpd.serve_forever()
```

2. Запустите `python3 app.py &`
3. Проверьте «сайт» `curl -v (Ваш домен)` . Изучите вывод команды.
4. Непосредственно на вашем компьютере откройте в браузере домен. Пройгнорируйте предупреждение браузера и посмотрите данные о сертификате. Изучите его структуру.
5. Убейте процесс запущенный на втором шаге.



А как в энтерпрайзе?

В лабораторной работе вы сделали примитивный удостоверяющий центр. В организации такую роль, вероятно, будет исполнять веб сервер Microsoft Active Directory, который будет подписывать сертификаты для внутреннего контура компании. Для внешнего контура, например, сайта для пользователей, сертификат будет приобретаться у глобальных организаций, реже использоваться Let's Encrypt.

Вопросы

1. Почему было предупреждение в браузере при открытии домена?
2. Что могло бы произойти, если бы Вы однопользователю передавали ключ?
3. В общих чертах, как происходит шифровка данных с помощью сертификата?
4. Почему система доверяет сертификатам, выписанными коммерческими организациями?

Понятийный минимум

1. Симметричное и асимметричное шифрование. Примеры.
2. Односторонняя функция. Хэширование.
3. Атака "Человек по середине".
4. Верификация сертификата. Цепь доверия. Корневой центр сертификации.
5. Установка TLS-сессии.
6. Принципы работы DNS.
7. Протокол Диффи — Хеллмана.